

Compile
2nd Year Specialty SIQ G2

Rapport

Titre:

Implémentation d'un
Analyseur Syntaxique LL(K)
en utilisant Javacc

Réalisé par

Rahal Nour Elhouda

Bayadh Hadjer

SIQ2

1. Conception

1.1. Conctruction de l'analyseur syntaxique descendant LL pour reconnaître le langage L

Soit la grammaire $\mathbf{G} = \langle N, T, S, P \rangle$ avec P :

$$\begin{aligned} S &\rightarrow A @ B \mid B \& A \mid \varepsilon \\ A &\rightarrow B \$ \mid S = S \\ B &\rightarrow A id \mid id S \end{aligned}$$

Rendre la grammaire propre

- **Vérifier les symboles inutiles :**

Un symbole est *inutile* s'il n'apparaît jamais dans la production d'une chaîne de terminaux.

Dans notre grammaire, nous remarquons que tous les symboles apparaissent dans la production de chaînes de terminaux. Ainsi, il n'y a aucun symbole à éliminer.

- **Éliminer les productions unitaires :**

Une production unitaire est une règle de la forme $A \rightarrow B$, où A et B sont des non-terminaux.

Dans notre grammaire, nous remarquons qu'il n'existe aucune règle de ce type. Ainsi, aucune élimination n'est nécessaire.

- **Éliminer les productions ε inutiles :**

- La règle $S \rightarrow \varepsilon$ doit être supprimée.
- Ajouter un nouveau non-terminal Z et introduire la production $Z \rightarrow S$, ainsi que $Z \rightarrow \varepsilon$.
- Ajouter des règles où toutes les occurrences de S peuvent être remplacées par ε .

La grammaire propre de $\mathbf{G} = \langle N, T, S, P \rangle$ avec P :

$$\begin{aligned} Z &\rightarrow S \mid \varepsilon \\ S &\rightarrow A @ B \mid B \& A \\ A &\rightarrow B \$ \mid S = S \mid S = \mid = S \mid = \\ B &\rightarrow A id \mid id S \mid id \end{aligned}$$

Eliminer le recursivité à gauche

on prend l'ordre suivant : $S > A > B$

- Pour S : on pas de recurivité direct donc on a rien a changé
- Pour A :
 - on remplace les ouccurence de S à la gauche de production de A par les production de S Resultat :

$$A \rightarrow B\$ \mid A@B = \mid B\&A = \mid A@B = S \mid B\&A = S \mid = S \mid =$$

- en suite on enleve recursivité à gauche direct :

$$\begin{aligned} A &\rightarrow B\$A' \mid B\&A = A' \mid B\&A = SA' \mid = SA' \mid = A' \\ A' &\rightarrow @B = A' \mid @B = SA' \mid \varepsilon \end{aligned}$$

–

- Pour B :
 - on remplace les ouccurence de A à la gauche de production de B par les productions de A Resultat :

$$B \rightarrow B\$A' id \mid B\&A = A' id \mid B\&A = SA' id \mid = SA' id \mid = A' id \mid id S \mid id$$

- en suite on enleve recursivité à gauche direct :

$$\begin{aligned} B &\rightarrow = SA' idB' \mid = A' idB' \mid id SB' \mid idB' \\ B' &\rightarrow \$A' idB' \mid \&A = A' idB' \mid \&A = SA' idB' \mid \varepsilon \end{aligned}$$

- Gramaire sans recursivité à gauche :

$$\begin{aligned} Z &\rightarrow S \mid \varepsilon \\ S &\rightarrow A @ B \mid B\&A \\ A &\rightarrow B\$A' \mid B\&A = A' \mid B\&A = SA' \mid = SA' \mid = A' \\ A' &\rightarrow @B = A' \mid @B = SA' \mid \varepsilon \\ B &\rightarrow = SA' idB' \mid = A' idB' \mid id SB' \mid idB' \\ B' &\rightarrow \$A' idB' \mid \&A = A' idB' \mid \&A = SA' idB' \mid \varepsilon \end{aligned}$$

Rendre la grammaire Factorisée à gauche

$$\begin{aligned}
 Z &\rightarrow S \mid \varepsilon \\
 S &\rightarrow A @ B \mid B \& A \\
 A &\rightarrow B X \mid = X' \\
 X &\rightarrow \$A' \mid \&A = X' \\
 X' &\rightarrow SA' \mid A' \\
 A' &\rightarrow @B = X' \mid \varepsilon \\
 B &\rightarrow = Y \mid id Y' \\
 Y &\rightarrow SA' idB' \mid A' idB' \\
 Y' &\rightarrow SB' \mid B' \\
 B' &\rightarrow \$A' idB' \mid \&A = Y \mid \varepsilon
 \end{aligned}$$

1.2. Construction de la table d'analyse

les ensembles début et suivant

Non-terminal	First	Follow
Z	=,id, ε	#
S	=,id	# , id , @ ,&,\$,=
A	=,id	# , id , @ ,&,\$,=
X	\$, &	# , id , @ ,&,\$,=
X'	=,id,@, ε	# , id , @ ,&,\$,=
A'	@, ε	# , id , @ ,&,\$,=
B	=, id	# , id , @ ,&,\$,=
Y	=,id,@, ε	# , id , @ ,&,\$,=
Y'	=,id,\$,&, ε	# , id , @ ,&,\$,=
B'	\$,&, ε	# , id , @ ,&,\$,=

table d'analyse

	=	id	\$	@	&	#
Z	$Z \rightarrow S$	$Z \rightarrow S$				$Z \rightarrow \varepsilon$
S	$S \rightarrow A@B$ $S \rightarrow B\&A$	$S \rightarrow A@B$ $S \rightarrow B\&A$				
A	$A \rightarrow BX$ $A \rightarrow = X'$	$A \rightarrow BX$				
A'	$A' \rightarrow \varepsilon$	$A' \rightarrow \varepsilon$	$A' \rightarrow \varepsilon$	$A' \rightarrow @B = X'$ $A' \rightarrow \varepsilon$	$A' \rightarrow \varepsilon$	$A' \rightarrow \varepsilon$
X			$X \rightarrow \$A'$		$X \rightarrow \&A' = X'$	
X'	$X' \rightarrow SA'$ $X' \rightarrow A'$	$X' \rightarrow SA'$ $X' \rightarrow A'$	$X' \rightarrow A'$	$X' \rightarrow A'$	$X' \rightarrow A'$	$X' \rightarrow A'$
B	$B \rightarrow = Y$	$B \rightarrow idY'$				
B'	$B' \rightarrow \varepsilon$	$B' \rightarrow \varepsilon$	$B' \rightarrow \$A'idB'$ $B' \rightarrow \varepsilon$	$B' \rightarrow \varepsilon$	$B' \rightarrow \&A = Y$ $B' \rightarrow \varepsilon$	$B' \rightarrow \varepsilon$
Y	$Y \rightarrow SA'idB'$	$Y \rightarrow SA'idB'$ $Y' \rightarrow A'idB'$		$Y \rightarrow SA'idB'$		
Y'	$Y' \rightarrow SB'$ $Y' \rightarrow B'$	$Y' \rightarrow SB'$ $Y' \rightarrow B'$	$Y' \rightarrow B'$	$Y' \rightarrow B'$	$Y' \rightarrow B'$	$Y' \rightarrow B'$

Après l'établissement de la table d'analyse pour $k = 1$ (nombre de termes lus en une seule fois), nous remarquons qu'elle est **multidéfinie**, ce qui signifie qu'elle ne satisfait pas la condition suivante :

Vérifier que pour chaque production du type

$$\text{DEBUT}(a.\text{SUIVANT}(A)) \cap \text{DEBUT}(b.\text{SUIVANT}(B)) = \text{vide}$$

Face à cette situation, deux solutions sont possibles :

- **Attribuer des priorités et adopter des conventions** pour les règles qui se retrouvent dans la même case. Cependant, nous constatons que certains mots appartenant au langage risquent d'être perdus avec ce changement. Par exemple, dans le cas de la production $S \rightarrow B\&A$, si nous la supprimons, nous perdrons le mot $T8_+Kg$, ce qui montre que cette solution n'est pas viable.
- **Augmenter la valeur de k** , c'est-à-dire examiner un plus grand nombre de symboles avant de prendre une décision.

Après plusieurs essais, nous avons constaté que pour $k = 2$ et $k = 3$, la grammaire **n'est toujours pas LL(k)**. En revanche, en augmentant k à 4, la grammaire satisfait les conditions nécessaires pour être un **analyseur syntaxique LL(4)**.

Nous allons donc implémenter un analyseur syntaxique **LL(4)** afin de lever les ambiguïtés et de garantir un parsing correct des chaînes du langage.

1.3. l'arbre syntaxique du mot de EXP : =@id

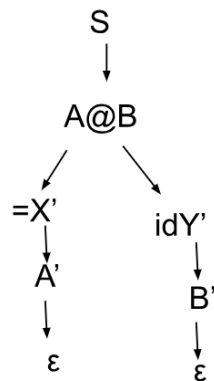


Figure 1: arbre syntaxique du mot de EXP : =@id

Conclusion

Dans ce travail, nous avons étudié l'analyse syntaxique des langages formels en nous concentrant sur les grammaires de type LL(k). Malgré les transformations classiques visant à rendre la grammaire plus exploitable—telles que l'élimination de la récursivité gauche et la factorisation—nous avons constaté que des ambiguïtés persistent.

Ces ambiguïtés se manifestent principalement dans la table d'analyse, où certaines entrées restent multidéfinies, ce qui empêche l'utilisation d'un analyseur LL(1). Nous avons alors exploré différentes approches pour résoudre ce problème, notamment l'attribution de priorités et de conventions, mais celles-ci se sont révélées insuffisantes, car elles entraînaient la perte de certains mots du langage.

Face à cette difficulté, nous avons adopté une autre solution : **augmenter la valeur de k dans LL(k)**. Après plusieurs essais, nous avons constaté que pour $k = 2$ et $k = 3$, l'ambiguïté persistait encore. Cependant, en portant k à 4, nous avons obtenu une grammaire qui satisfait les contraintes d'un analyseur LL(4).

L'implémentation de cet analyseur LL(4) nous a permis de lever les ambiguïtés et de garantir une reconnaissance correcte des chaînes du langage. Ce travail met en évidence l'importance du paramètre k dans l'analyse LL et montre que, dans certains cas, il est nécessaire d'augmenter la profondeur du *lookahead* pour assurer une analyse syntaxique déterministe et sans ambiguïté.